

№4 ПРАКТИКАЛЫҚ ЖҰМЫС

Қарсылас жағдайындағы іздеу және ойын ағаштары

Студенттерге арналған әдістемелік нұсқаулық | Жасанды интеллект негіздері | Python

1. Мақсаты және оқу нәтижелері

Мақсаты: жеткізу қызметтері арасындағы ойдан шығарылған бәсеке мысалында Minimax және Alpha-Beta бұтақтарды қию алгоритмдерін қолдану, Python тілінде жүрістерді қарастыру ретінің әсерін тексеру және тереңдігі шектелген іздеуде салмақталған бағалау функциясын пайдалану.

- Терминалдық түйіндердің мәндері арқылы ішкі түйіндердің мәндерін есептеу (Minimax, төменнен жоғары қарай).
- Түбірден бастап Alpha-Beta алгоритмінің орындалуын қадағалау: түйінге кірген кездегі α және β мәндерін жазу және қарастырылмай қалған бұтақтарды анықтау.
- Рекурсивті Minimax пен Alpha-Beta алгоритмдерін іске асыру, рекурсивті шақыруларды санау және ағашты қарастыру ретін салыстыру.
- Салмақталған белгілер бойынша Eval(s) мәнін есептеу және салмақтар өзгергенде таңдалатын күйдің неге өзгертінін түсіндіру.
- Қосымша: Expectimax көмегімен CHANCE түйіндеріндегі күтілетін мәндерді есептеу.

2. Жағдаят және кіріспе тапсырмалар

Астана-Алматы бағыты бойынша екі ойдан шығарылған жеткізу компаниясы бәсекелеседі. Біздің ЖИ агентіміз — MAX, ал бәсекелес — MIN. Бастапқы стратегияларымыз: А — төмен баға, В — стандартты жеткізу, С — жедел жеткізу. Бәсекелес бағаны төмендету, өзгеріссіз қалдыру немесе жеткізуді жылдамдату арқылы жауап бере алады. Төмендегі әр нәтиже — біздің компанияның миллион теңгемен берілген шартты пайдасы. Бұл — оқытуға арналған жеңілдетілген нөлдік қосынды моделі.

MAX стратегиясы	Бағаны төмендету	Өзгеріссіз	Жеткізуді жылдамдату
А: Төмен баға	8	6	4
В: Стандартты жеткізу	3	7	5
С: Жедел жеткізу	2	9	1

Кіріспе тапсырма 1. Әр стратегияның ең төменгі пайдасын табыңыз. Содан кейін осы минимумдардың ішіндегі ең үлкені бар стратегияны таңдаңыз. Есептеулеріңізді көрсетіп, ең қолайсыз жағдай туралы болжамды түсіндіріңіз.

Кіріспе тапсырма 2. Осы қарапайым ағашты Alpha-Beta әдісімен солдан оңға қарай қарастырыңыз. А стратегиясын бағалағаннан кейін В және С тармақтарының қалған жапырақтарын қарастыру қажет пе, анықтаңыз. Бағаланған және қиылған жапырақтар санын көрсетіңіз.

3. 1-тапсырма — тереңірек ағашта Minimax қолдану

Төмендегі берілген ағашты пайдаланыңыз. А — MAX; В және С — MIN; D, E, F және G — MAX. H–O — терминалдық мәндер. Бұл ағаш үш стратегиясы бар кіріспе тапсырмадан бөлек қарастырылады.

A(MAX) → B(MIN) → D(MAX) → H:4, I:9
 ↳ E(MAX) → J:7, K:2
 ↳ C(MIN) → F(MAX) → L:3, M:8
 ↳ G(MAX) → N:6, O:1

Тапсырмалар: (a) D, E, F және G мәндерін есептеңіз; (b) B және C мәндерін есептеңіз; (c) A мәнін тауып, түбірден таңдалатын тармақты белгілеңіз; (d) рекурсивті іздеу түбірден басталса да, мәндер неліктен төменнен жоғары қарай есептелетінін түсіндіріңіз.

Ішкі түйін	D	E	F	G	B	C	A
Minimax мәні	9	7	8	6	7	6	7

4. 2-тапсырма — Alpha–Beta орындалуын қадағалау

Осы тереңірек ағашты тереңдік бойынша іздеу арқылы алдымен солдан оңға, кейін оңнан солға қарай қарастырыңыз. Бастапқыда $\alpha = -\infty$ және $\beta = +\infty$: α — MAX үшін осы уақытқа дейін табылған ең жақсы төменгі шек, β — MIN үшін ең жақсы жоғарғы шек. MAX түйінінде α , MIN түйінінде β жаңартылады. Егер $\alpha \geq \beta$ болса, осы түйіннің қалған балаларын қарастыру тоқтатылады.

Маңызды: «кірудегі α » және «кірудегі β » мәндерін түйінге кірген сәтте жазыңыз, түйіннің соңғы мәні анықталғаннан кейін емес. Қарастырылған балаларды нақты өту ретімен көрсетіңіз. Қайтарылатын мән қарастырылған балалар өңделгеннен кейін ғана толтырылады.

4.1 Солдан оңға қарай қадағалау

Түйін	Түрі	Кірудегі α	Кірудегі β	Қарастырылған балалар	Қайтарылған мән	Қиылғандар
A	MAX	-шексіздік	+шексіздік	B, C	7	Жоқ
B	MIN	-шексіздік	+шексіздік	D, E	7	Жоқ
D	MAX	-шексіздік	+шексіздік	H, I	9	Жоқ
E	MAX	-шексіздік	9	J, K	7	Жоқ
C	MIN	7	+шексіздік	F, G	6	Жоқ
F	MAX	7	+шексіздік	L, M	8	Жоқ
G	MAX	7	8	N, O	6	Жоқ

4.2 Оңнан солға қарай қадағалау

Түйін	Түрі	Кірудегі α	Кірудегі β	Қарастырылған балалар	Қайтарылған мән	Қиылғандар
A	MAX	-шексіздік	+шексіздік	C, B	7	Жоқ
C	MIN	-шексіздік	+шексіздік	G, F	6	Жоқ
G	MAX	-шексіздік	+шексіздік	O, N	6	Жоқ
F	MAX	-шексіздік	6	M	8	L
B	MIN	6	+шексіздік	E, D	7	Жоқ
E	MAX	6	+шексіздік	K, J	7	Жоқ
D	MAX	6	7	I	9	H

Екі қадағалаудан кейін әр бағытта қарастырылмаған жапырақтарды дөңгелектеп белгілеңіз. Түбірдің соңғы мәндерін және қарастырылған түйіндер санын салыстырыңыз. Қиылған жапырақты орындалған рекурсивті шақыру ретінде санамаңыз.

5. 3-тапсырма — Python тіліндегі эксперимент

Атаулары берілген тереңірек ағашты сөздік (dictionary) арқылы іске асырыңыз. Дәрістері `is_terminal(state)`, `successors(state)` және `utility(state)` көмекші функцияларын пайдаланыңыз. `minimax(state, maximizing, counter, reverse=False)` және `alphabeta(state, alpha, beta, maximizing, counter, reverse=False)` функцияларын іске асырыңыз. Терминалдық жапырақтарды қоса алғанда, әрбір рекурсивті шақыруды санаңыз. Әр ішкі түйіндегі балалар ретін кері өзгерту үшін `reverse=True` қолданыңыз.

Ағашты ұсынудың ұсынылатын нұсқасы:

```
tree = {
    "A": ["B", "C"], "B": ["D", "E"], "C": ["F", "G"],
    "D": ["H", "I"], "E": ["J", "K"],
    "F": ["L", "M"], "G": ["N", "O"]
}
payoffs = {"H": 4, "I": 9, "J": 7, "K": 2,
           "L": 3, "M": 8, "N": 6, "O": 1}

def is_terminal(state):
    return state in payoffs

def successors(state, reverse=False):
    children = tree[state]
    return list(reversed(children)) if reverse else children

def utility(state):
    return payoffs[state]
```

А қосымшасындағы толық, жеке іске қосылатын Python бағдарламасын орындаңыз. Оның көмекші функцияларын және рекурсивті Minimax пен Alpha-Beta іске асырылуын зерттеңіз. Содан кейін салыстыру кестесіне өз бақылауларыңызды енгізіп, нәтижелерді түсіндіріңіз.

Өту бағыты	Minimax мәні	Minimax шақырулары	Alpha-Beta мәні	Alpha-Beta шақырулары	Қиылған жапырақтар
Солдан оңға	7	15	7	15	Жоқ(0)
Оңнан солға	7	15	7	13	L, H (2)

Талдау: қай өту реті көбірек бұтақты қиюға мүмкіндік береді? Жүрістерді қарастыру реті түбірдегі дұрыс Minimax мәнін өзгерте ме? Жауабыңызды өзіңіз санаған түйіндер санымен дәлелдеңіз.

6. 4-тапсырма — тереңдігі шектелген іздеудегі бағалау

Іздеу терминалдық күйге жетпей d тереңдігінде тоқтаса, тоқтау нүктесіндегі күйді үш шартты белгі арқылы бағалаңыз. Формула: $Eval(s) = w_1 \times Margin + w_2 \times Reliability + w_3 \times Capacity$, мұнда Margin — пайда маржасы, Reliability — сенімділік, Capacity — өткізу мүмкіндігі. Барлық белгілер салыстырмалы бір шкалада берілген оқу мәндері.

Күй	Пайда маржасы	Сенімділік	Өткізу мүмкіндігі
X	8	7	2
Y	5	5	8

А бөлімі. Бастапқы салмақтарды (0.5, 0.3, 0.2) қолданыңыз. $Eval(X)$ және $Eval(Y)$ есептеңіз. MAX қай күйді таңдайды?

В бөлімі. Өткізу мүмкіндігінің салмағын 0.5-ке дейін арттырып, қалған екі салмақты пропорционалды өзгертіңіз: (0.3125, 0.1875, 0.5). Екі күйдің бағасын қайта есептеңіз. Таңдалатын күй өзгере ме және неліктен? Әр көбейту мен қосуды көрсетіңіз.

Салмақтар	$Eval(X)$	$Eval(Y)$	Таңдалатын күй
-----------	-----------	-----------	----------------

Бастапқы (0.5, 0.3, 0.2)	6,5	5,6	X
Өзгертілген (0.3125, 0.1875, 0.5)	4,8125	6,5	Y

7. Қосымша тапсырма — Expectimax

2-бөлімдегі A/B/C жеткізу стратегияларының қарапайым кестесіне қайта оралыңыз. Бұл балама модельде MIN орнына CHANCE түйінін қолданыңыз: нәтижелерді қарсылас әдейі таңдамайды, олардың белгілі ықтималдықтары бар. Ықтималдықтар пайда кестесіндегі үш бағанға сол ретпен сәйкес келеді.

Стратегия	P(бағаны төмендету)	P(өзгеріссіз)	P(жылдам жеткізу)	Күтілетін пайда
A	0.2	0.3	0.5	5.4 млн теңге
B	0.2	0.5	0.3	5,6 млн теңге
C	0.7	0.2	0.1	3,3 млн теңге

A, B және C үшін $V(\text{CHANCE}) = \sum p_i \times V(\text{child}_i)$ мәнін есептеңіз. Қай стратегияның күтілетін пайдасы ең жоғары? Оны кіріспе тапсырмадағы Minimax таңдауымен салыстырыңыз. Ықтималдық моделін қандай жағдайда қолдануға болатынын қысқаша түсіндіріңіз.

8. Есепті тапсыру талаптары

- Белгілері қойылған тереңірек ойын ағашы және төменнен жоғары қарай орындалған Minimax есептеулері.
- Екі өту бағыты үшін толтырылған Alpha–Beta қадағалау кестелері: қарастырылған және қиылған балалар көрсетілуі керек.
- Python бастапқы коды, нәтиже скриншоттары немесе консоль шығысының көшірмесі және рекурсивті шақырулар саны берілген салыстыру кестесі.
- Салмақтарды өзгерткенге дейінгі және өзгерткеннен кейінгі бағалау функциясының есептеулері.
- Бақыланған нәтижелерге негізделген қысқаша қорытынды; орындалған жағдайда Expectimax есептеулері.

Есептің ұсынылатын құрылымы: жағдаятты өз сөзіңізбен түсіндіру → Ағаш/сызба → Есептеулер және қадағалау → Python коды мен нәтижесі → Салыстыру → Қорытынды. Жұмысты өз бетіңізше орындаңыз; бөлек берілетін 2-тапсырма (Assignment 2) өзінің бастапқы деректері бойынша дербес шешілуі тиіс.

9. Ойлануға арналған сұрақтар

- Alpha–Beta алгоритмі азырақ түйінді қарастырса да, неліктен сол шешімді қайтара алады?
- Рекурсивті шақырулар саны нені көрсетеді және нені көрсетпейді?
- Қарсыластың жүрісін болжау міндетті түрде қарсыласты түсіну дегенді білдіре ме?

А ҚОСЫМШАСЫ — Толық іске қосылатын Python бағдарламасы: Minimax және Alpha-Beta

practical4_game_tree.py атауымен сақтаңыз. Іске қосу: python3 practical4_game_tree.py. Бұл деректерді, барлық көмекші функцияларды, екі алгоритмді, main() функциясын, қарастырылған түйіндерді тіркеуді және екі бағыттағы нәтижелерді қамтитын дербес бағдарлама.

```
# practical4_game_tree.py
# Толық бағдарлама: Minimax, Alpha-Beta және жүрістерді қарастыру реті
```

```
# 1. Ойын ағашы мен терминалдық мәндерді сақтау.
```

```
tree = {
    "A": ["B", "C"],
    "B": ["D", "E"], "C": ["F", "G"],
    "D": ["H", "I"], "E": ["J", "K"],
    "F": ["L", "M"], "G": ["N", "O"]
}
payoffs = {
    "H": 4, "I": 9, "J": 7, "K": 2,
    "L": 3, "M": 8, "N": 6, "O": 1
}
```

```
# 2. Дәрістергі көмекші функциялар.
```

```
def is_terminal(state):
    return state in payoffs

def successors(state, reverse=False):
    children = tree[state]
    return list(reversed(children)) if reverse else children

def utility(state):
    return payoffs[state]
```

```
# 3. Minimax: MAX және MIN кезектесіп рекурсивті шақырылады.
```

```
def minimax(state, maximizing, counter, reverse=False):
    counter[0] += 1 # Count every call, including terminal nodes.
    if is_terminal(state):
        return utility(state)

    if maximizing:
        value = float("-inf")
        for child in successors(state, reverse):
            child_value = minimax(child, False, counter, reverse)
            value = max(value, child_value)
        return value
    else:
        value = float("inf")
        for child in successors(state, reverse):
            child_value = minimax(child, True, counter, reverse)
            value = min(value, child_value)
        return value
```

```
# 4. Alpha-Beta: alpha >= beta болса, қалған балаларды қию.
```

```
def alphabeta(state, alpha, beta, maximizing, counter,
              visited, pruned, reverse=False):
    counter[0] += 1
    visited.append(state)
    if is_terminal(state):
        return utility(state)

    children = successors(state, reverse)
    if maximizing:
        value = float("-inf")
        for index, child in enumerate(children):
```

```

        child_value = alphabeta(
            child, alpha, beta, False, counter,
            visited, pruned, reverse
        )
        value = max(value, child_value)
        alpha = max(alpha, value)
        if alpha >= beta:
            # Only children not explored are marked as pruned.
            pruned.extend(children[index + 1:])
            break
    return value
else:
    value = float("inf")
    for index, child in enumerate(children):
        child_value = alphabeta(
            child, alpha, beta, True, counter,
            visited, pruned, reverse
        )
        value = min(value, child_value)
        beta = min(beta, value)
        if alpha >= beta:
            pruned.extend(children[index + 1:])
            break
    return value

# 5. Негізгі бағдарлама: екі өту ретін де орындау.
def main():
    for reverse in (False, True):
        order = "right-to-left" if reverse else "left-to-right"
        mm_calls = [0]
        ab_calls = [0]
        visited = []
        pruned = []

        mm_value = minimax("A", True, mm_calls, reverse)
        ab_value = alphabeta(
            "A", float("-inf"), float("inf"), True,
            ab_calls, visited, pruned, reverse
        )

        print("\nTraversal:", order)
        print("Minimax root:", mm_value, "calls:", mm_calls[0])
        print("Alpha-Beta root:", ab_value, "calls:", ab_calls[0])
        print("Alpha-Beta visited:", " -> ".join(visited))
        print("Pruned leaves:", pruned if pruned else "none")
        assert mm_value == ab_value

if __name__ == "__main__":
    main()

```

В ҚОСЫМШАСЫ — Толық іске қосылатын Python бағдарламасы: бағалау функциясы

practical4_evaluation.py атауымен сақтаңыз. Іске қосу: python3 practical4_evaluation.py. Белгілер салмақтарын өзгерткенге дейін және кейін таңдалатын күйлерді салыстырыңыз.

```
# practical4_evaluation.py
# Толық бағдарлама: тереңдік шегіндегі салмақталған бағалау.
```

```
features = {
    "X": {"margin": 8, "reliability": 7, "capacity": 2},
    "Y": {"margin": 5, "reliability": 5, "capacity": 8}
}
```

```
def evaluate(state, weights):
    f = features[state]
    return (weights[0] * f["margin"]
            + weights[1] * f["reliability"]
            + weights[2] * f["capacity"])
```

```
def main():
    experiments = {
        "Initial": (0.5, 0.3, 0.2),
        "Capacity increased": (0.3125, 0.1875, 0.5)
    }
    for name, weights in experiments.items():
        assert abs(sum(weights) - 1.0) < 1e-9
        scores = {state: evaluate(state, weights)
                  for state in features}
        preferred = max(scores, key=scores.get)
        print(name, "weights:", weights)
        print("Eval(X):", scores["X"])
        print("Eval(Y):", scores["Y"])
        print("MAX prefers:", preferred, "\n")
```

```
if __name__ == "__main__":
    main()
```

С ҚОСЫМШАСЫ — Толық іске қосылатын Python бағдарламасы: Expectimax (қосымша)

practical4_expectimax.py атауымен сақтаңыз. Іске қосу: python3 practical4_expectimax.py. Күтілетін пайданы Minimax алгоритмінің ең қолайсыз жағдайдағы таңдауымен салыстырыңыз.

```
# practical4_expectimax.py
# Қосымша толық бағдарлама: жеткізу стратегиялары үшін CHANCE түйіндері.
```

```
payoffs = {
    "A": [8, 6, 4],
    "B": [3, 7, 5],
    "C": [2, 9, 1]
}
probabilities = {
    "A": [0.2, 0.3, 0.5],
    "B": [0.2, 0.5, 0.3],
    "C": [0.7, 0.2, 0.1]
}

def chance_value(strategy):
    p = probabilities[strategy]
    values = payoffs[strategy]
    assert abs(sum(p) - 1.0) < 1e-9
    return sum(prob * payoff for prob, payoff in zip(p, values))

def main():
    expected = {strategy: chance_value(strategy)
                 for strategy in payoffs}
    worst_case = {strategy: min(values)
                  for strategy, values in payoffs.items()}
    for strategy in payoffs:
        print(strategy, "expected payoff:", round(expected[strategy], 2),
              "million KZT")
    print("Expectimax chooses:", max(expected, key=expected.get))
    print("Minimax chooses:", max(worst_case, key=worst_case.get))

if __name__ == "__main__":
    main()
```